FRONTEND DESIGN DOCUMENT

SimuFlow

Interactive System Design Visualiser

v1.0 · March 2026

React + TS	ReactFlow	MobX	Web Workers
Core Stack	Canvas Engine	State Layer	Simulation

1. Product Vision & Design Principles

SimuFlow is a browser-based system design environment where engineers build architectures, run simulations, and watch their systems behave — or break — in real time. It serves two audiences equally: engineers learning distributed systems concepts and working engineers stress-testing real architectural decisions.

> The central bet: the canvas is not a drawing surface. It is a live, running simulation environment. Every design decision — from particle animation to node colour states to named chaos scenarios — must reinforce that the system in front of you is alive.

1.1 Design Principles

- Instant gratification — simulation starts in < 2 seconds, no loading spinners, no wasm compilation walls
- Named over generic — every chaos scenario has a real engineering name, not a generic 'inject failure' button
- Both audiences feel at home — a student learning why caches matter and a senior engineer validating a microservices topology should both find value
- The canvas tells the story — node health and traffic are readable from the canvas itself, not only from a separate panel
- Share-first — every saved diagram is one click from a shareable read-only link

1.2 Competitive Position

Capability	Static tools (Miro/draw.io)	Paperdraw	SimuFlow
Traffic simulation	None	Yes	Yes — particle-animated
Node health visualisation	None	Basic	Colour + pulse ring, real-time
Named chaos scenarios	None	Generic injection	6 named real-world scenarios
Time to interactive	Instant	~60s (wasm load)	< 2s (Web Worker, JS)
Share diagrams	Export image	Unknown	Read-only shareable URL + fork
Audience	Anyone	Interview prep	Both: learners + practitioners

2. Technology Stack

Package	Version	Purpose	Rationale
react	^18	UI framework	Concurrent mode, Suspense for async node loading
typescript	^5	Type safety	Node/edge/frame schemas must be airtight — see §4
reactflow	^11	Canvas engine	Custom nodes, edge types, pan/zoom, minimap — all native
mobx	^6	Simulation state	Observable maps update metrics panel with zero boilerplate
mobx-react-lite	^3	React bindings	observer() makes components auto-reactive to store changes
framer-motion	^11	Particle animation	GPU-accelerated, path-following animation for request particles
tailwindcss	^3	Styling	Utility-first, no runtime overhead
recharts	^2	Metric sparklines	Lightweight reactive chart library
@supabase/supabase-js	^2	Auth + persistence	Save/load/share diagrams
zustand	^4	UI-only state	Panel open/close, modals — keeps MobX stores clean
vite	^5	Build tool	Native Web Worker support, fast HMR
nanoid	^5	ID generation	Short collision-resistant node/edge IDs

3. Frontend Architecture

3.1 Directory Structure

Path	Contents
src/types/topology.ts	Shared type contract — every cross-boundary interface and enum. See §4.
src/components/canvas/	ReactFlow host, custom node components, ParticleEdge
src/components/panels/	ConfigPanel, MetricsPanel, ChaosPanel, Toolbar
src/components/library/	Node palette sidebar, preset blueprint cards
src/components/ui/	Shared design system: buttons, sliders, badges, tooltips
src/stores/	GraphStore, SimulationStore, ChaosStore, UIStore
src/workers/simulation.worker.ts	The full simulation loop — imports from engine/ only
src/workers/engine/	requestGenerator, graphTraverser, constraintSolver, metricAggregator, particleEmitter
src/workers/chaos/	One file per named chaos scenario — each implements ChaosScenario interface
src/presets/	JSON blueprints: web_app, cached_web_app, microservices, queue_system
src/hooks/	useWorkerBridge, useSimulation, useNodeConfig, useShareLink
src/lib/supabase/	Client factory, auth hooks, diagram CRUD, share token logic

3.2 Component Hierarchy

Component	Responsibility	Key state source
<App />	Root, auth gate, routing	Supabase auth session
<WorkspaceLayout />	Three-panel + toolbar shell	UIStore.panels
<CanvasPanel />	ReactFlow host, drop target	GraphStore.nodes/edges
<NodeLibrary />	Left sidebar — draggable node palette	Static config
<ConfigPanel />	Right sidebar — selected node properties	GraphStore.selectedNode
<MetricsPanel />	Bottom panel — live stats + sparklines	SimulationStore.metrics
<ChaosPanel />	Chaos scenario selector and controls	ChaosStore.active

<SimulationToolbar />	Play/pause/stop, speed dial, share button	SimulationStore.status
<CustomNode />	Per-type node — health ring, metric chips, handles	SimulationStore.nodeStates
<ParticleEdge />	Animated edge with flowing request particles	SimulationStore.edgeFlows
<ShareModal />	Generate/copy shareable read-only URL	useShareLink hook

4. Shared Type Contract — src/types/topology.ts

topology.ts is the single most important file in the codebase. It defines every data structure that crosses a process boundary: frontend canvas ↔ Web Worker, frontend ↔ backend API, and backend ↔ Supabase. Any field in the backend Pydantic models must exist here first.

> Rule: topology.ts imports nothing from React, MobX, or any UI library. It is pure TypeScript — interfaces, enums, and type aliases only. The Web Worker imports directly from this file. The backend Pydantic models in app/models/ mirror every field defined here.

4.1 Versioning & Enums

```
export const TOPOLOGY_VERSION = "1.0" as const;

export enum NodeType {
  Client       = "client",
  LoadBalancer = "load_balancer",
  ApiServer    = "api_server",
  Cache        = "cache",
  Database     = "database",
  Queue        = "queue",
  CDN          = "cdn",
  Microservice = "microservice",
}

export enum LBStrategy {
  RoundRobin       = "round_robin",
  LeastConnections = "least_connections",
  Random           = "random",
}

export enum NodeHealth {
  Idle       = "idle",       // simulation not running
  Healthy    = "healthy",    // utilisation < 60%
  Stressed   = "stressed",   // utilisation 60–90%
  Bottleneck = "bottleneck", // utilisation > 90%
  Failed     = "failed",     // chaos killed node or failure rate triggered
}

export enum ChaosScenarioId {
  DbFailover     = "DB_FAILOVER",
  CacheStampede  = "CACHE_STAMPEDE",
  NetPartition   = "NET_PARTITION",
  TrafficSpike   = "TRAFFIC_SPIKE",
  Cascade        = "CASCADE",
  SlowDependency = "SLOW_DEP",
}

export enum SimulationStatus {
  Idle    = "idle",
```

```
  Running = "running",
  Paused  = "paused",
  Chaos   = "chaos", // running + at least one active chaos scenario
}
```

4.2 Node Config — Discriminated Union

Each NodeType has its own config interface. The NodeConfig discriminated union means TypeScript narrows config fields when you switch on node.nodeType — no casting needed anywhere in the codebase.

```
export interface BaseNodeConfig {
  capacity:    number; // max rps before node saturates
  latencyMs:   number; // processing latency p50 in ms
  failureRate: number; // probability 0-1 that a request fails
  timeoutMs:   number; // requests exceeding this emit an error event
}

export interface ClientConfig {
  rps:   number;  // outbound requests per second from this source node
  burst: boolean; // whether client generates burst patterns
}

export interface LoadBalancerConfig extends BaseNodeConfig {
  strategy: LBStrategy;
  replicas: number;
}

export interface CacheConfig extends BaseNodeConfig {
  hitRate: number; // 0-1. On miss, request falls through to backing store
}

export interface QueueConfig {
  delayMs:  number; // processing delay per message in ms
  maxDepth: number; // 0 = unlimited. Above this, messages are dropped
}

// Discriminated union — narrow with node.nodeType before accessing node.config
export type NodeConfig =
  | { nodeType: NodeType.Client;       config: ClientConfig }
  | { nodeType: NodeType.LoadBalancer; config: LoadBalancerConfig }
  | { nodeType: NodeType.ApiServer;    config: BaseNodeConfig }
  | { nodeType: NodeType.Cache;        config: CacheConfig }
  | { nodeType: NodeType.Database;     config: BaseNodeConfig }
  | { nodeType: NodeType.Queue;        config: QueueConfig }
  | { nodeType: NodeType.CDN;          config: CacheConfig }
  | { nodeType: NodeType.Microservice; config: BaseNodeConfig };
```

4.3 Topology — The Diagram Data Model

TopologySchema is what gets saved to Supabase and loaded back into the canvas. SimNode and SimEdge are what GraphStore holds and ReactFlow renders.

```
export interface CanvasPosition { x: number; y: number; }
export interface CanvasViewport { x: number; y: number; zoom: number; }

// SimNode = id + label + position + NodeConfig discriminated union
export type SimNode = {
  id:       string;         // nanoid — unique within diagram
  label:    string;         // user-set display name
  position: CanvasPosition;
} & NodeConfig;

export interface SimEdge {
  id:       string; // nanoid
  sourceId: string;
  targetId: string;
  label?:   string; // optional edge label shown at idle
}

export interface TopologySchema {
  version:  typeof TOPOLOGY_VERSION;
  nodes:    SimNode[];
  edges:    SimEdge[];
  viewport: CanvasViewport;
}
```

4.4 Simulation Runtime State

These types are never stored in Supabase. They exist only in the browser — SimulationStore and the Web Worker — during an active simulation session.

```
export interface NodeRuntimeState {
  nodeId:         string;
  health:         NodeHealth;
  utilisationPct: number;  // 0–100
  currentRps:     number;
  queueDepth:     number;
  errorRate:      number;  // rolling 5s window, 0–1
}

export interface EdgeFlowState {
  edgeId:        string;
  particleCount: number;  // capped at 12 — above that stroke width increases
  throughput:    number;  // actual rps on this edge
  errorRatio:    number;  // 0–1
  isPartitioned: boolean; // true when NET_PARTITION chaos affects this edge
}

export interface MetricSnapshot {
  timestamp:     number;
  throughput:    number;
  p50LatencyMs:  number;
```

```
  p95LatencyMs:  number;
  p99LatencyMs:  number;
  errorRate:     number;
  totalRequests: number;
}

export interface ChaosEvent {
  scenarioId:      ChaosScenarioId;
  phase:           "activated" | "escalated" | "resolved";
  affectedNodeIds: string[];
  timestamp:       number;
}
```

4.5 SimulationFrame — The Worker Heartbeat

Posted from the Web Worker to the main thread every ~100ms. useWorkerBridge consumes every frame and calls SimulationStore.absorbFrame() in a MobX runInAction batch. This is the only path by which SimulationStore is updated.

```
export interface SimulationFrame {
  tickId:        number;
  timestamp:     number;
  nodeStates:    Record<string, NodeRuntimeState>;
  edgeFlows:     Record<string, EdgeFlowState>;
  globalMetrics: MetricSnapshot;
  chaosEvents:   ChaosEvent[];
  bottlenecks:   string[]; // node IDs at > 90% utilisation
}
```

4.6 Worker Message Protocol

All messages between the main thread and simulation.worker.ts are typed here. The Worker's onmessage handler exhaustively switches over WorkerInboundMessage.type — TypeScript will error if a case is missing.

```
export type WorkerInboundMessage =
  | { type: "START";           topology: TopologySchema; speed: number }
  | { type: "PAUSE" }
  | { type: "RESUME" }
  | { type: "STOP" }
  | { type: "SET_SPEED";       speed: number }
  | { type: "UPDATE_TOPOLOGY"; topology: TopologySchema }
  | { type: "ACTIVATE_CHAOS";  scenario: ActiveScenario }
  | { type: "DEACTIVATE_CHAOS"; instanceId: string };

export type WorkerOutboundMessage =
  | { type: "FRAME"; frame: SimulationFrame }
  | { type: "READY" }
  | { type: "ERROR"; message: string };
```

4.7 Chaos Types

```
export interface ChaosScenarioDef {
  id:               ChaosScenarioId;
  name:             string;
  tag:              string;
  description:      string;
  validTargetTypes: NodeType[]; // empty = no target needed (e.g. TrafficSpike)
  requiresTarget:   boolean;
  requiresConfig:   boolean;   // e.g. spike multiplier must be set before
activation
}

export interface ActiveScenario {
  id:            string;                  // nanoid instance ID
  scenarioId:    ChaosScenarioId;
  targetNodeIds: string[];
  config:        Record<string, unknown>; // e.g. { multiplier: 5,
durationSeconds: 30 }
  activatedAt:   number;
  resolvesAt?:   number;                  // undefined = manual resolve only
}
```

4.8 API Response Types

These mirror the backend Pydantic response models. camelCase on the frontend, snake_case on the backend — transformed automatically by Pydantic's alias_generator. See Backend doc §8 for the full mirror mapping.

```
export interface DiagramSummary {
  id:        string;
  name:      string;
  isPublic:  boolean;
  createdAt: string; // ISO 8601
  updatedAt: string;
}

export interface DiagramResponse extends DiagramSummary {
  topology:   TopologySchema;
  shareToken: string | null;
  forkCount:  number;
}

export interface DiagramListResponse {
  items:    DiagramSummary[];
  total:    number;
  page:     number;
  pageSize: number;
}
```

```
export interface ShareResponse {
  shareUrl:   string;
  shareToken: string;
  isPublic:   true;
}

export interface ForkResponse { diagramId: string; name: string; }

export interface PresetBlueprint {
  slug:        string;
  name:        string;
  description: string;
  category:    "fundamentals" | "patterns";
  topology:    TopologySchema;
}
```

5. State Management

5.1 GraphStore (MobX)

Source of truth for the canvas data model. Never mutated by the simulation — it owns topology, not runtime state. All types here come from topology.ts.

Observable	Type	Description
nodes	Map<id, SimNode>	All canvas nodes keyed by ID
edges	Map<id, SimEdge>	All canvas edges keyed by ID
selectedNodeId	string | null	Drives ConfigPanel open/content
diagramId	string | null	Supabase diagram ID if saved
diagramName	string	Editable name shown in toolbar
isDirty	boolean	Unsaved changes indicator

- Actions: addNode, removeNode, updateNodeConfig, connectNodes, disconnectEdge, loadPreset, loadFromSupabase, saveToSupabase, setName
- Computed: nodeCount, edgeCount, sourceNodes (no incoming edges), terminalNodes

5.2 SimulationStore (MobX)

Owns all runtime state. Updated exclusively by useWorkerBridge — no component writes to this store directly.

Observable	Type	Description
status	SimulationStatus	Current simulation phase — from topology.ts enum
nodeStates	Map<id, NodeRuntimeState>	Utilisation %, queue depth, current rps, health
edgeFlows	Map<id, EdgeFlowState>	Particle count, throughput, error ratio per edge
globalMetrics	MetricSnapshot	Throughput, p50/p95/p99 latency, error rate
metricsHistory	MetricSnapshot[]	Rolling 60s window for sparklines
speed	0.25|0.5|1|2|4	Simulation speed multiplier
tickCount	number	Total ticks elapsed
elapsedSeconds	number	Wall-clock seconds since last START — incremented by main thread timer, pauses when simulation pauses

- Actions: start, pause, resume, stop, reset, setSpeed, absorbFrame(frame: SimulationFrame)

- Computed: bottleneckNodes, systemHealthScore (0–100), hasErrors

5.3 ChaosStore (MobX)

Observable	Type	Description
activeScenarios	Map<id, ActiveScenario>	Currently running chaos instances
scenarioLog	ChaosEvent[]	Timestamped record of all chaos events
availableScenarios	ChaosScenarioDef[]	Full catalogue of all 6 named scenarios

- Actions: activateScenario, deactivateScenario, clearAll, resolveScenario
- Computed: isChaosModeActive, affectedNodeIds

5.4 UIStore (Zustand)

- panelState: { left: boolean, right: boolean, bottom: boolean, chaos: boolean }
- activeModal: 'share' | 'preset' | 'settings' | null
- theme: 'dark' | 'light' | onboardingComplete: boolean

5.5 Worker ↔ Store Bridge

> useWorkerBridge is the single point of contact between the Worker and React. It creates the Worker, subscribes to messages, and calls SimulationStore.absorbFrame() in a MobX batch. Nothing else touches the Worker directly.

1. useWorkerBridge instantiates simulation.worker.ts on workspace mount
2. GraphStore topology change (MobX reaction) → bridge posts WorkerInboundMessage START or UPDATE_TOPOLOGY
3. ChaosStore change → bridge posts ACTIVATE_CHAOS or DEACTIVATE_CHAOS
4. SimulationStore.speed change → bridge posts SET_SPEED
5. Worker posts FRAME → bridge calls absorbFrame() inside runInAction batch
6. On unmount → bridge posts STOP and terminates Worker

6. Canvas & Node System

6.1 ReactFlow Configuration

Config	Value	Reason
nodeTypes	Custom map per type	Each type is a full React component
edgeTypes	{ default: ParticleEdge }	All edges animate particles during simulation
snapToGrid	true — 20px	Clean aligned architectural diagrams
connectionMode	loose	Any handle connects to any handle
selectNodesOnDrag	false	Prevent accidental multi-select while panning
fitView	true on preset load	Auto-center when a blueprint is loaded
deleteKeyCode	Delete / Backspace	Standard keyboard UX
minZoom / maxZoom	0.2 / 2	See full architecture or zoom into one node

6.2 Node Catalogue

Node Type	Default Config	Simulation Role
Client / User	RPS: 100, burst: false	Traffic source — no capacity limit
Load Balancer	Cap: 5000 rps, strategy: round-robin	Distributes traffic across downstream nodes
API Server	Cap: 500 rps, latency: 20ms, fail: 0%	Core processing node
Cache	Cap: 5000 rps, latency: 2ms, hit rate: 80%	Short-circuits requests on hit; miss falls through
Database	Cap: 200 rps, latency: 80ms, fail: 0%	Terminal node; slowest by design
Message Queue	No cap, delay: 50ms, maxDepth: 0	Buffers traffic; decouples producers/consumers
CDN	Cap: 10000 rps, latency: 5ms, hit: 85%	Edge cache; absorbs static asset traffic
Microservice	Cap: 300 rps, latency: 30ms, fail: 1%	Generic service with configurable deps

6.3 Node Component Anatomy

- Header row: type icon (16px) + editable label + type badge chip
- Health ring: animated SVG border — colour and pulse speed driven by NodeRuntimeState.health
- Metric chips (simulation only): current rps / utilisation% / queue depth — hidden at idle
- Handles: top, bottom, left, right — all active ReactFlow connection handles
- Click: opens ConfigPanel for this node in the right sidebar

> Health ring: Green = utilisation < 60%. Yellow = 60–90% (pulse speed increases linearly). Red = > 90% or failure triggered (fast pulse + glow). Grey = idle. Transitions are CSS variable-driven for smooth animation.

7. Simulation Engine

7.1 Why Web Worker

The simulation loop runs entirely inside a Web Worker — no DOM access, no React dependency. Pure TypeScript logic that imports only from topology.ts and engine/. This keeps the main thread free for 60fps canvas rendering at all times.

> Performance target: canvas renders at 60fps always. Worker posts frames every 100ms. Particle count capped at 12 per edge — above that, edge stroke width increases instead. This keeps animation smooth even under extreme simulated load.

7.2 Engine Modules

Module	File	Responsibility
Request Generator	engine/requestGenerator.ts	Produces N requests per tick via Poisson distribution at configured RPS
Graph Traverser	engine/graphTraverser.ts	BFS from source nodes — routes each request through connected nodes to terminal
Constraint Solver	engine/constraintSolver.ts	Applies per-node capacity, latency sampling (normal dist), failure rate, queue overflow
Metric Aggregator	engine/metricAggregator.ts	Rolling 5s window — computes throughput, p50/p95/p99, error rate, node utilisation
Particle Emitter	engine/particleEmitter.ts	Derives per-edge particle count and velocity from request flow rate
Chaos Modifier	chaos/chaosModifier.ts	Intercepts constraint solving to apply active scenario overrides

7.3 Simulation Tick Loop

7. Worker receives START message (WorkerInboundMessage) with full TopologySchema + speed
8. setInterval fires every 100ms, adjusted by speed multiplier (e.g. 4x = 25ms interval)
9. requestGenerator produces requests for this tick at configured RPS
10. chaosModifier checks active scenarios and modifies node constraints accordingly
11. graphTraverser routes each request from source nodes to terminal nodes, recording path
12. constraintSolver applies constraints at each hop — latency sampled, capacity checked, failures rolled
13. metricAggregator accumulates tick data into rolling windows

14. particleEmitter derives edge particle deltas from throughput per edge
15. Worker posts SimulationFrame to main thread via postMessage

8. Chaos Scenarios

Chaos scenarios are named, real-world failure and stress patterns. Each is a distinct, learnable event — not a generic slider. They run on top of the normal simulation; the user can activate one while the system is already running and watch behaviour change in real time.

> Design rule: every scenario must be (1) named after the real engineering term, (2) visually distinct on the canvas when active, (3) teach something specific about distributed systems. If it doesn't teach anything, it's not worth shipping.

8.1 The Six Scenarios

Database Failover / Replica Lag *[DB_FAILOVER]*

The primary database node is killed mid-simulation. Traffic that was flowing to it now has two paths: some nodes retry and hit a replica (with added lag), others fail immediately. Models the classic 'primary goes down mid-deployment' scenario.

Engine behaviour: *Primary DB node: capacity → 0, failure rate → 100%. Replica node (if connected): latency += 200–400ms. Requests routed to primary emit error events. Bottleneck highlight appears. Scenario resolves after configurable TTR.*

Cache Stampede / Cold Start *[CACHE_STAMPEDE]*

The cache is cleared or restarted cold. All traffic that would have hit the cache falls through to the database simultaneously. The DB saturates instantly. Engineers learn why cache warming and probabilistic early expiry matter.

Engine behaviour: *Cache node: hit rate → 0% for duration. All requests bypass cache and route to backing DB. DB utilisation spikes sharply — typically triggers bottleneck within 2–3 ticks. Cache hit rate recovers to configured value over 30s. Edge between cache and DB glows red.*

Network Partition *[NET_PARTITION]*

A partition is inserted between two user-selected node groups. Requests that cross the partition fail silently. Models split-brain — a fundamental CAP theorem concept. Partitioned edges render dashed with a red X overlay.

Engine behaviour: *Selected edges: isPartitioned → true. Requests attempting to cross partition: failure rate → 100%. Partitioned edges render dashed. Nodes on isolated side show degraded state. Partition resolves when user clicks Resolve.*

Traffic Spike / Thundering Herd *[TRAFFIC_SPIKE]*

Incoming traffic is multiplied by a configurable factor (3x, 5x, 10x) for a duration. Models a viral moment, flash sale, or DDoS burst. Shows which node saturates first and whether the cache absorbs the spike.

Engine behaviour: *Client node RPS: multiplied by spike factor. Spike ramps up over 5s, holds for configurable duration, ramps down over 5s. Bottleneck nodes emerge naturally based on topology. Toolbar shows spike factor and countdown timer.*

Cascading Failure *[CASCADE]*

One node is stressed past capacity. When it drops requests, upstream nodes queue up, backpressure propagates, and the failure spreads — the 'retry storm'. Engineers learn why circuit breakers and exponential backoff exist.

Engine behaviour: *Target node: capacity reduced to 60% of current traffic. Upstream nodes: on receiving errors, apply retry multiplier (1.5x additional load to target). Failure propagates upstream. Canvas shows cascade visually — red state travels outward from target.*

Slow Dependency *[SLOW_DEP]*

One node's latency spikes dramatically — a slow external API, a locked DB query, or a GC pause. Upstream nodes queue up even though they themselves are healthy. Teaches why timeouts, circuit breakers, and async patterns matter.

Engine behaviour: *Target node: latency multiplied by 8–15x (configurable). Node stays alive — no failures, just slow. Queue depths build upstream. Health ring shifts to yellow/red based on queue depth. Timeout threshold triggers error events if latency exceeds it.*

8.2 ChaosPanel UI

- Scenario catalogue: 6 cards with name, one-line description, and Activate button
- Active scenarios section: running scenarios with elapsed time, Resolve button, effect summary
- Event log: last 20 timestamped chaos events — activation, escalation, resolution
- Target selector: for scenarios needing a target (Slow Dep, DB Failover, Partition) — node/edge picker appears inline before activation
- Spike configurator: for Traffic Spike — multiplier selector (3x / 5x / 10x) and duration before activation

9. Particle Rendering

9.1 ParticleEdge Component

All edges use the custom ParticleEdge type. At idle only the base path renders. During simulation, particles animate along the path using Framer Motion.

Particle type	Colour	Trigger
Normal request	Blue #3B82F6	Standard traffic flowing through edge
Error / failed	Red #EF4444	Request that failed at source or destination node
Cache hit	Green #10B981	Request served from cache — never reaches DB
Queued / slow	Amber #F59E0B	Request waiting in queue at downstream node
Partitioned	None — particles stop	Edge is network-partitioned

9.2 Animation Implementation

- Each particle is a Framer Motion motion.circle element following SVG path via pathLength 0→1 animation
- Duration is inversely proportional to edge throughput — more traffic = faster particles
- Object pool of 200 pre-allocated particle elements — recycle via opacity, not DOM creation/destruction
- Cap: 12 visible particles per edge. Above 12, stroke width increases (2px → 8px max)
- Partitioned edges: dashed stroke + red X icon at midpoint, no particles render

10. UI Panels & Layout

10.1 Layout Zones

Zone	Dimensions	Content	Collapsible
Top Toolbar	Full width, 56px	Logo, sim controls, diagram name, share button	No
Left Sidebar	280px	Node palette + preset blueprint cards	Yes
Center Canvas	Flex fill	ReactFlow canvas with minimap (bottom-right corner)	—
Right Sidebar	320px	ConfigPanel (node selected) or ChaosPanel (chaos tab)	Yes
Bottom Panel	280px height	MetricsPanel — live stats, sparklines, node breakdown, event log	Yes

10.2 Node Configuration Panel

- Node name — editable inline text field
- Type badge — read-only, colour-matched to node type
- Capacity (req/sec) — number input with range slider
- Latency p50 (ms) — number input with range slider
- Failure rate (%) — range slider 0–100
- Cache / CDN additionally: hit rate (%) slider
- Load balancer additionally: strategy selector (round-robin / least-connections / random)
- Advanced (collapsed): timeout threshold ms, replica count
- Danger zone: Delete node button with confirmation

10.3 Simulation Toolbar

Control	States / Options	Behaviour
Play	Idle → Running	Posts START to Worker with current topology
Pause	Running → Paused	Posts PAUSE; Worker holds state, particles freeze
Resume	Paused → Running	Posts RESUME; simulation continues from same tick
Stop	Any → Idle	Posts STOP; SimulationStore resets to zero
Speed dial	0.25x / 0.5x / 1x / 2x / 4x	Posts SET_SPEED; Worker adjusts tick interval

Share	Always available	Opens ShareModal — generates or copies shareable URL
Save	Available when dirty	Saves diagram to Supabase
Diagram name	Editable text	Inline edit, auto-saves on blur

10.4 Metrics Panel

- Global stats row: Throughput (rps), p50 / p95 / p99 Latency (ms), Error Rate (%)
- Sparkline row: recharts LineChart per metric over last 60s rolling window
- Node breakdown table: per-node utilisation bar, rps, queue depth, health badge
- Bottleneck alert: sticky banner when any node > 90% utilisation — names the node
- Chaos event log: last 20 events with timestamp, scenario name, affected node, status

10.5 Preset Blueprints

Preset	Topology	What it teaches
Basic Web App	Client → LB → 2× API Server → DB	Load balancing basics, single DB as bottleneck
Cached Web App	Client → LB → API Server → Cache → DB	Cache hit/miss flow, thundering herd risk
Microservices	Client → API GW → 3 services → shared DB + per-service cache	Inter-service latency, cascading failure paths
Queue-Based System	Client → API → Queue → 2× Workers → DB	Decoupling, backpressure, worker scaling

10.6 Canvas HUD Card

A small persistent card overlaid on the canvas — bottom-left corner, above the minimap. Always visible during simulation. Answers the question 'how long has this been running?' which is easy to lose track of on an open-ended simulation. Four fields, read-only, updates every second.

Field	Source	Description
RPS	SimulationStore.globalMetrics.throughput	Current system-wide requests per second — live from latest SimulationFrame
Elapsed	SimulationStore.elapsedSeconds	Wall-clock simulation time since last START, formatted as mm:ss. Pauses when simulation is paused.
Requests	SimulationStore.globalMetrics.totalRequests	Cumulative total requests processed since simulation started. Formatted with thousands separator.

Speed	SimulationStore.speed	Current speed multiplier — mirrors the toolbar dial. e.g. '2×'

- Component: <SimulationHUD /> — positioned absolute, bottom-left of canvas container, z-index above ReactFlow pane
- Reads only from SimulationStore — fully reactive via MobX observer(), updates every tick
- Hidden at idle (SimulationStatus.Idle) — fades in when simulation starts via Framer Motion AnimatePresence
- Compact dark card ~200px wide — does not obscure nodes; user can drag it if it overlaps
- Elapsed timer runs via a useEffect setInterval on the main thread — not derived from tickCount — so it reflects real wall-clock time regardless of speed multiplier

> Elapsed is wall-clock time, not simulated time. At 4x speed the system processes more requests per second of real time, but elapsed still counts real seconds. This keeps the HUD grounded in 'how long have I actually been watching this' rather than a confusing simulated clock.

TODO — Simulation Budget / End Condition

The HUD card has a natural extension: a Budget field that caps or scopes the simulation. This needs more thought before designing. Two directions to evaluate:

Option A — Raw request budget (paperdraw-style): user sets a number e.g. 10,000 requests; simulation auto-stops when totalRequests hits that number. Simple, reproducible, good for A/B comparisons ('run both architectures to 10k requests and compare p95').

Option B — Named end conditions: 'run until first bottleneck', 'run for 60 simulated seconds', 'run until chaos scenario resolves'. Feels more intentional and teaches something specific. Harder to implement.

Decision needed: which framing fits the product better, or do both live in different parts of the UI? Once decided, add Budget as a fifth field to the HUD card, wire it to a new simulationBudget observable in SimulationStore, and handle auto-stop logic in the Worker tick loop.

11. Share & Persistence

11.1 Save & Load

- Diagrams auto-save on simulation stop if user is signed in
- Manual save via toolbar Save button at any time
- Diagram list in top-left menu — shows name, last saved, thumbnail

11.2 Shareable Read-Only Links

- Share button opens ShareModal — calls POST /api/v1/diagrams/{id}/share
- Resulting URL: simuflow.app/view/{share_token}
- Shared view: read-only canvas — all nodes, edges, config visible, no editing
- 'Fork this diagram' button on shared view — saves copy into viewer's account
- Share revoked at any time from the same modal

> Fork is the seed of the community library. When someone forks a shared diagram it becomes theirs to edit and re-share. This creates a network of blueprints without needing to build a separate community system at launch.

12. Performance Strategy

12.1 Rendering

- MobX observer(): applied only to CustomNode, ParticleEdge, MetricsPanel — other components don't re-render on simulation ticks
- ReactFlow: nodesDraggable={false} during active simulation
- Metrics panel: recharts updates debounced to 500ms
- Particle pool: 200 pre-allocated Framer Motion instances, recycled via opacity, no GC pressure
- absorbFrame() runs inside MobX runInAction batch — one React reconciliation pass per frame

12.2 Bundle

- ReactFlow, recharts: dynamic import — only loaded when workspace route is active
- Web Worker: separate Vite entry point — excluded from main bundle entirely
- Target: < 200KB gzipped initial load; workspace chunk lazy-loaded on route

12.3 Worker Memory

- Frame history: circular buffer capped at 600 frames (60s at 10fps)
- SimulationFrame objects reused across ticks — fields overwritten in-place, no per-tick allocation
- Graph topology in Worker: plain JS objects, no class instances — faster structured clone

13. Testing Strategy

13.1 Unit Tests (Vitest)

- engine/graphTraverser: routing through linear, branching, cyclic, disconnected topologies
- engine/constraintSolver: capacity overflow, failure rate distribution, latency sampling bounds
- engine/metricAggregator: rolling window correctness, p95/p99 calculation accuracy
- chaos/chaosModifier: each scenario's constraint overrides — DB_FAILOVER sets capacity 0, SLOW_DEP multiplies latency
- GraphStore: add/remove/update nodes, computed values, preset loading
- SimulationStore: state machine transitions, absorbFrame correctness
- ChaosStore: scenario activation, deactivation, log appending

13.2 Component Tests (React Testing Library)

- CustomNode: correct health ring colour at utilisation 50%, 80%, 95%
- ConfigPanel: updates GraphStore on slider/input change
- ChaosPanel: activating DB_FAILOVER calls ChaosStore.activateScenario with correct params
- MetricsPanel: bottleneck banner appears at 91% utilisation

13.3 End-to-End (Playwright)

- Full flow: drag 3 nodes, connect edges, run simulation, verify particles animate, metrics update
- Preset load: select 'Cached Web App', run simulation, verify cache hit logic
- Chaos: activate Traffic Spike 5x, verify bottleneck banner appears within 5s
- Share: save diagram, click share, open URL in incognito, verify read-only canvas loads
- Auth: sign up, create diagram, sign out, sign in, verify diagram persists

14. Build Milestones

Milestone	Scope	Est.
M1 — Scaffold	Vite + React + TS + ReactFlow canvas, MobX stores, three-panel layout, Supabase auth	4 days
M2 — Node System	All 8 node types, drag-from-palette, ConfigPanel, health ring (static)	4 days
M3 — Worker Engine	Web Worker scaffold, all 5 engine modules, SimulationFrame, worker bridge hook	5 days
M4 — Simulation UI	Particle edge renderer, live health ring, toolbar play/pause/stop/speed	4 days
M5 — Chaos Scenarios	All 6 named scenarios, ChaosStore, ChaosPanel, chaosModifier, canvas visual states	5 days
M6 — Metrics Panel	Live stats, sparklines, bottleneck banner, event log, node breakdown table	3 days
M7 — Presets + Share	4 blueprint presets, save/load to Supabase, share token, read-only view, fork button	3 days
M8 — Polish	Onboarding, tooltips, responsive layout, performance audit, E2E tests	4 days

Total: ~32 engineering days. M2 (Node System) and M3 (Worker Engine) can run in parallel after M1. M5 (Chaos) is isolated enough to develop independently of M6 (Metrics).